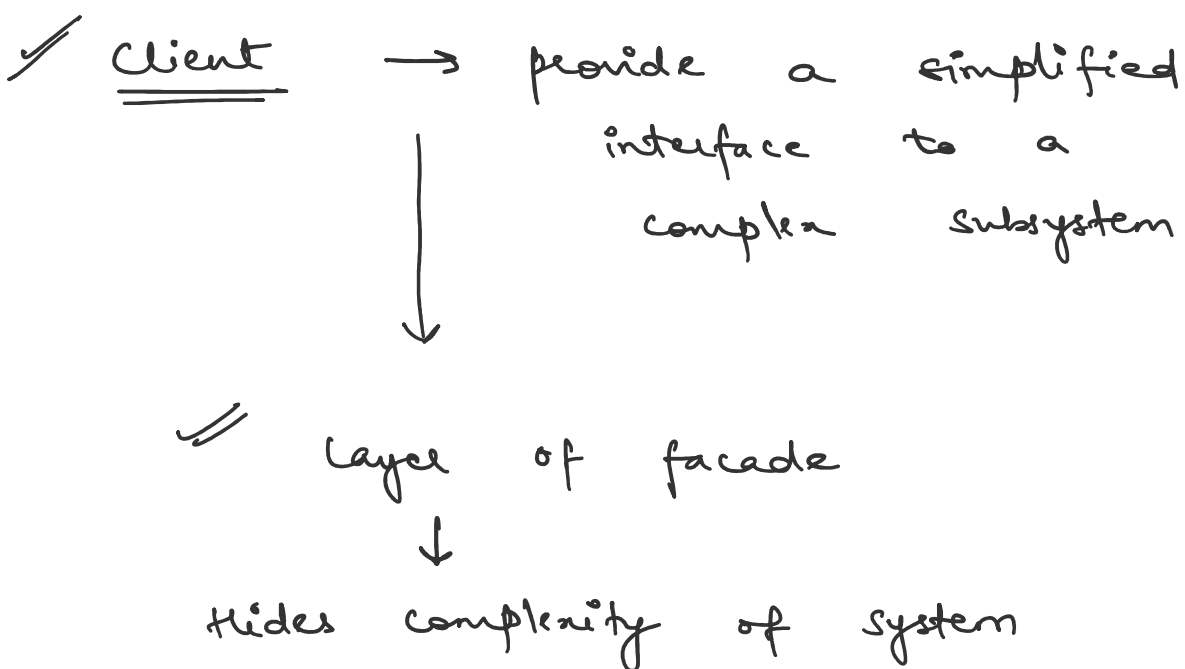
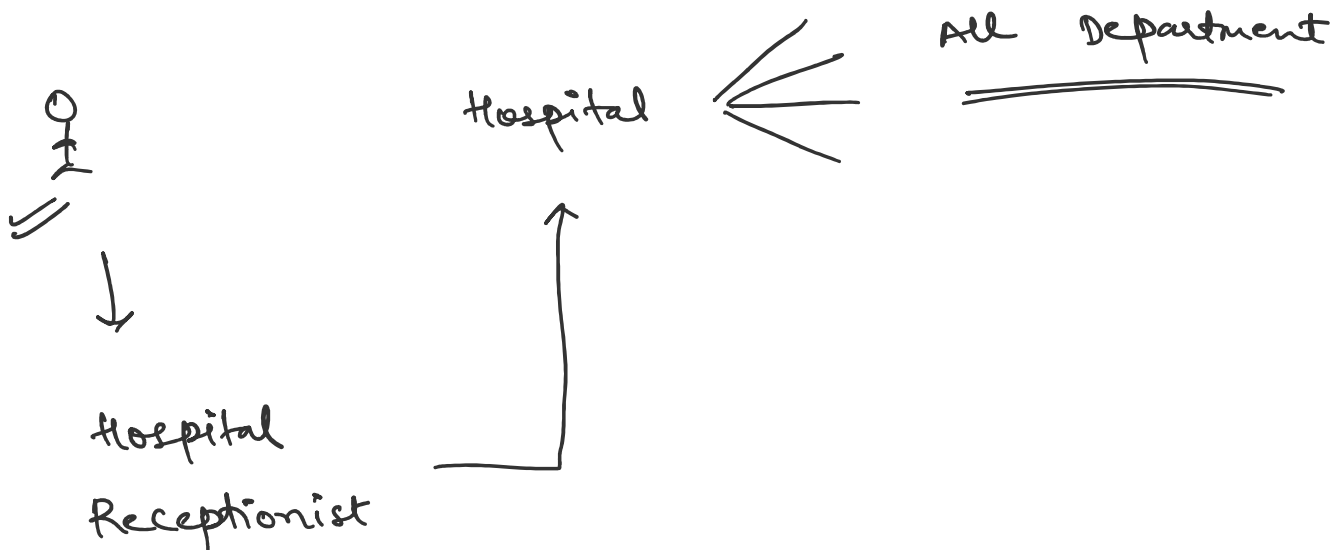


Class 14

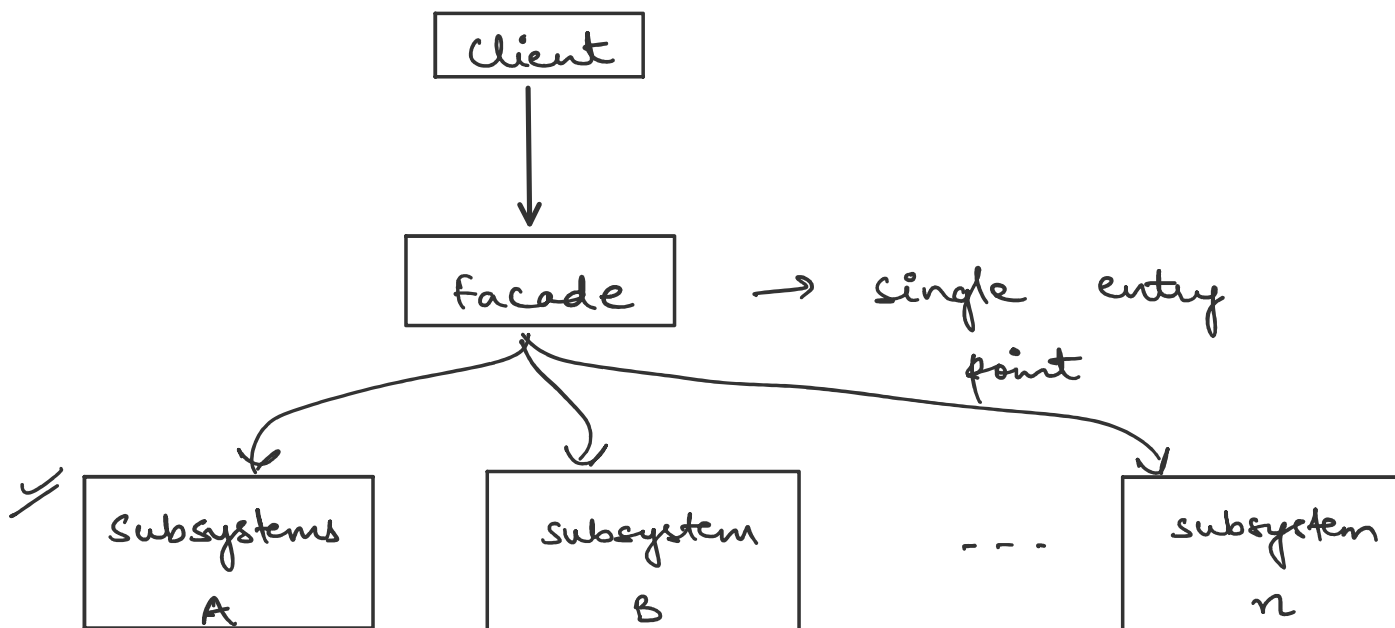
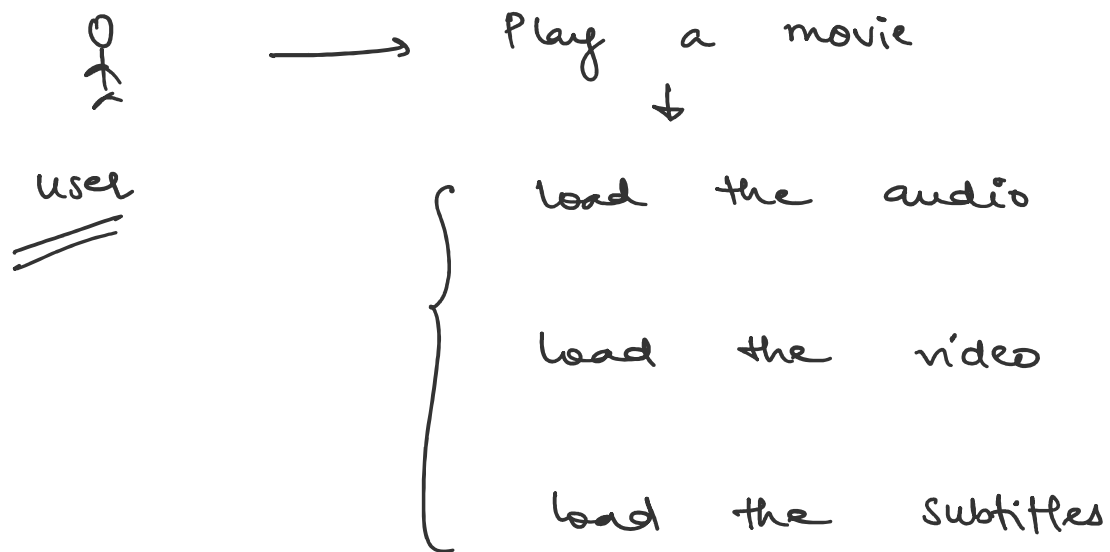
Monday, July 6, 2026

9:06 PM

Facade DP



and provides easy to use
interface.



Travel Booking system

- Booking the flight ✓
- Reserving the hotel ✓
- Booking a taxi ✓
- Purchase travel insurance
- Processing the payment
- Sending confirmation mail

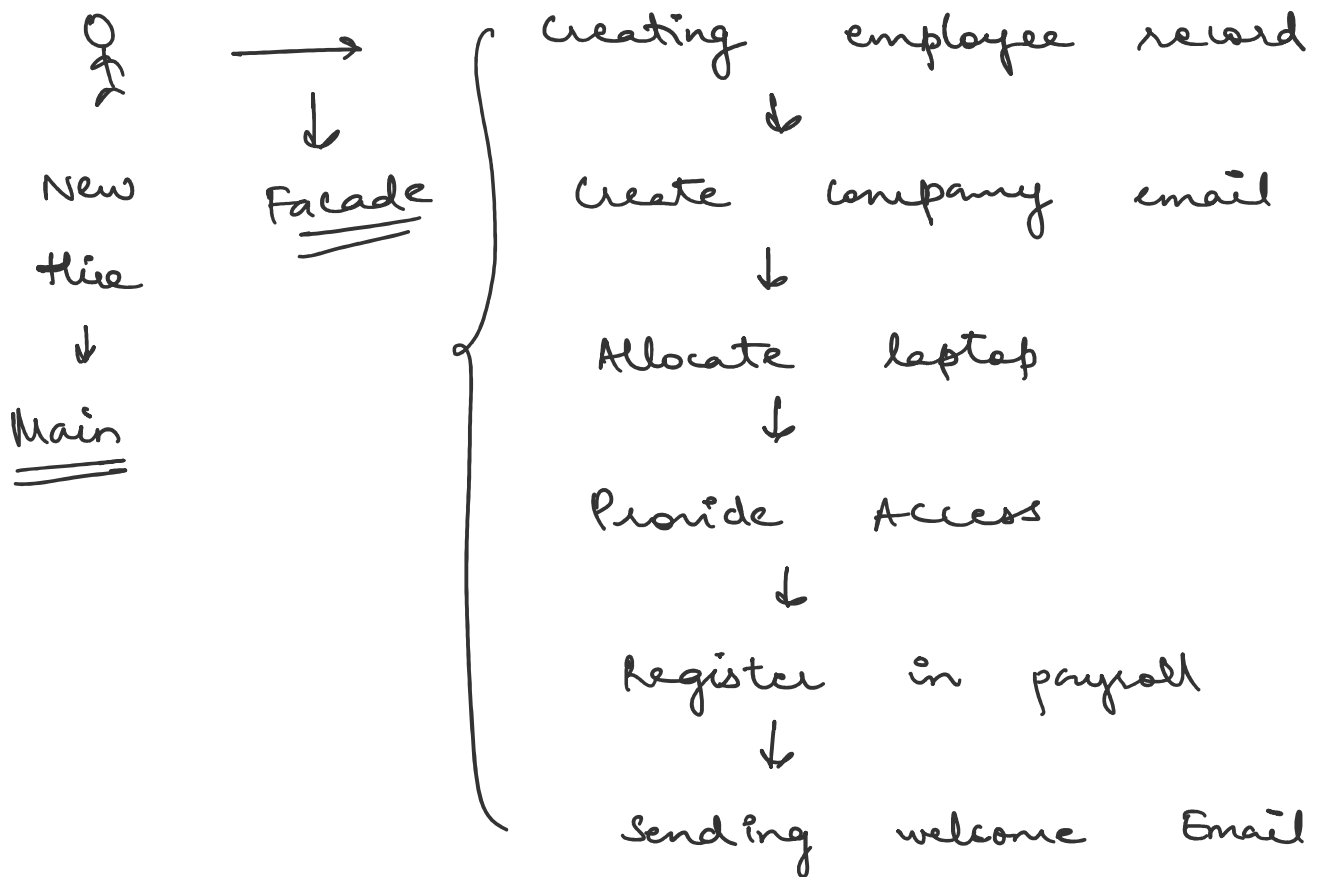


Booking . com



Booking
Package

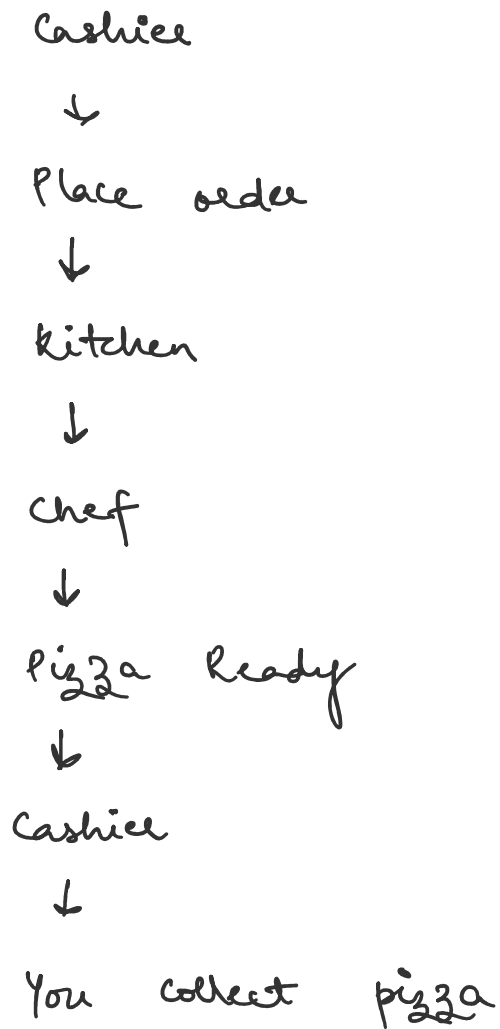
Employee onboarding system



Sequence Diagram

order pizza

You
↓



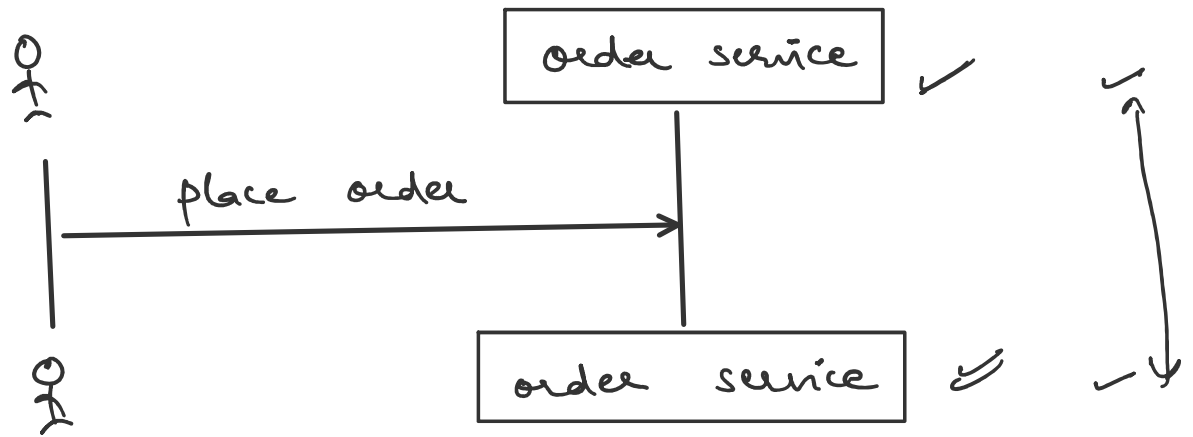
Main components of sequence diagram -

→ Actor

↓
Starting the process

 customer

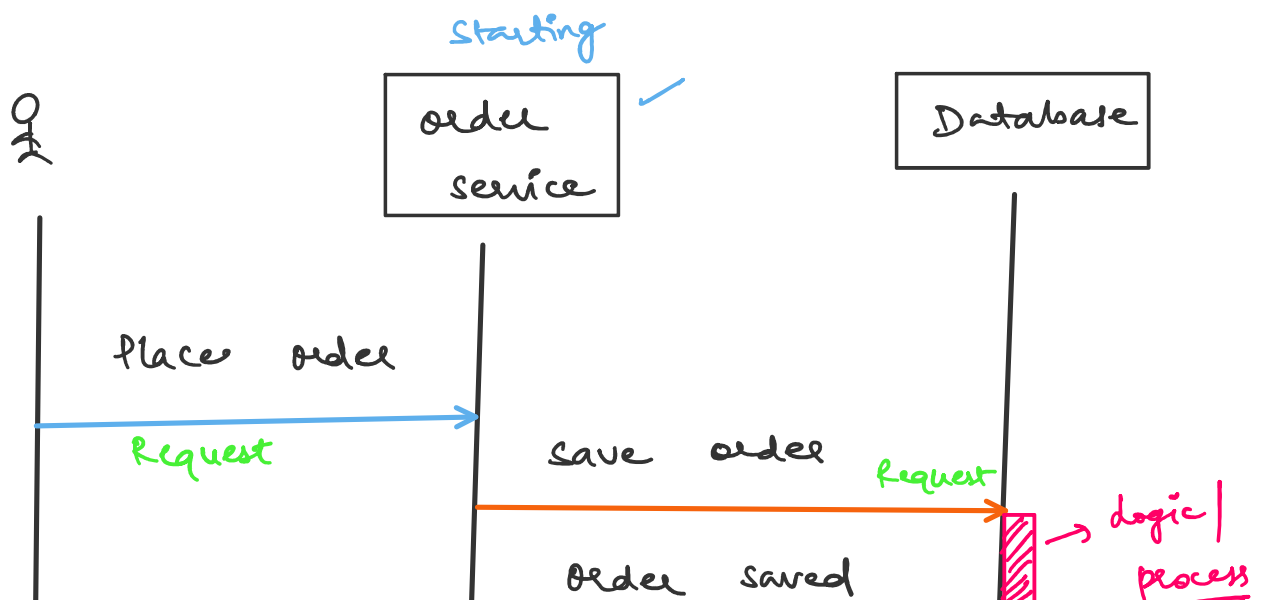
Customer places an order.

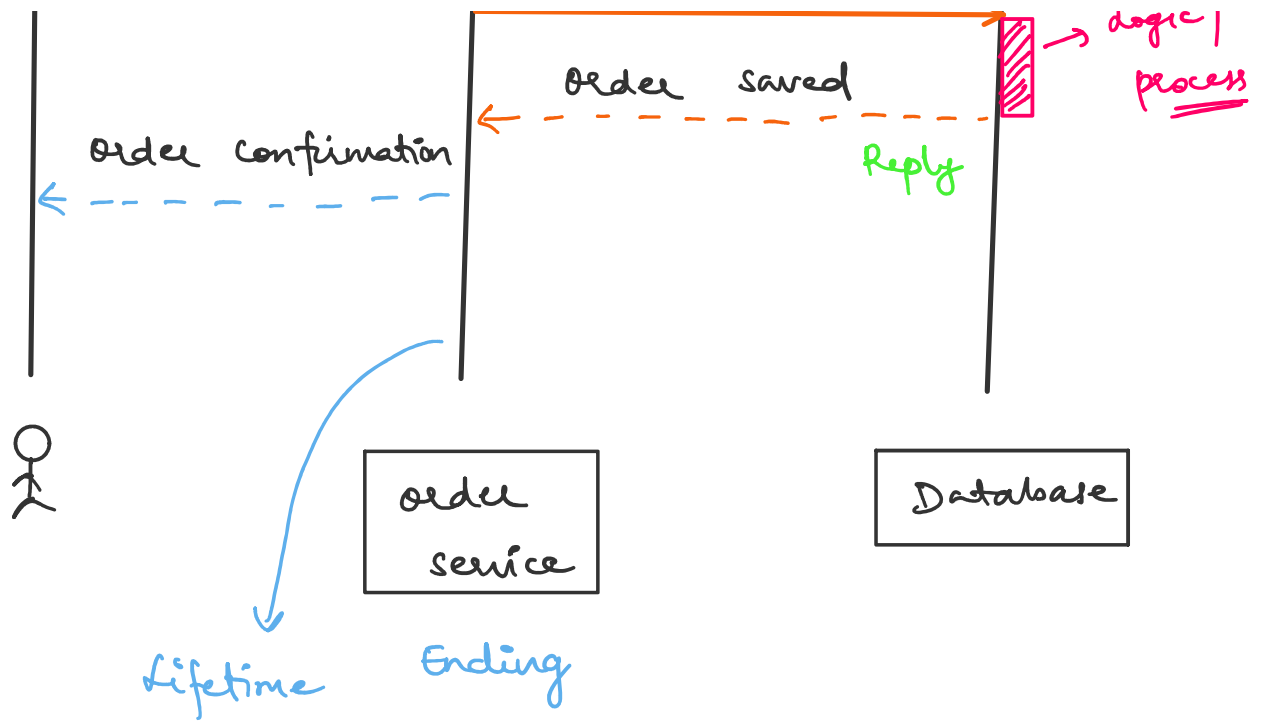


→ Object / Participant



object / class / service





Travel Eg -

Actor → client 

Participants → Travel Booking facade → Fa

Flight Service → FS

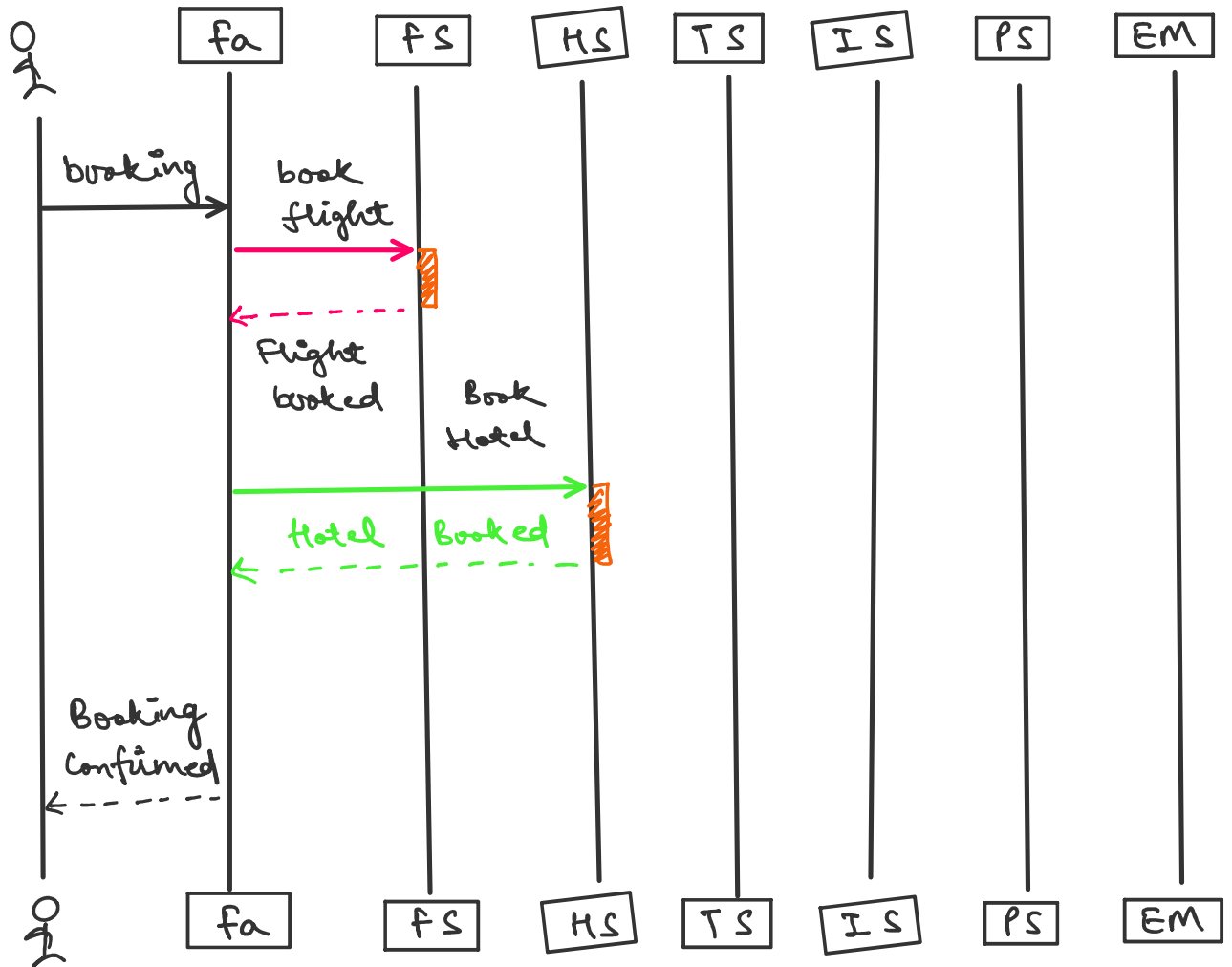
Hotel Service → HS

Taxi Service → TS

Insurance service → IS

Payment service → PS

Email service → ES



Notes :

Decorator Pattern

The Facade Pattern is a **structural design pattern** that provides a **simplified interface** to a complex subsystem.

Instead of allowing the client to interact with multiple classes directly, the Facade introduces a single entry point that coordinates all the required operations internally.

The client only communicates with the Facade, while the Facade communicates with the underlying subsystem classes.

In simple words, the Facade hides the complexity of a system and provides an easy-to-use interface for the client.

Why Do We Need the Facade Pattern?

As applications grow, a single business operation often requires multiple classes to work together.

For example, completing one task may require:

- Validation
- Database operations
- External API calls
- Notifications
- Logging
- Report generation

If every client directly communicates with all these classes:

- The client becomes tightly coupled to many classes.
- The client must understand the execution sequence.
- Code becomes difficult to maintain.
- Changes in subsystem classes affect every client.

The Facade Pattern solves this problem by introducing a single class that coordinates all these operations.

The client only interacts with the Facade, while the Facade manages the complexity behind the scenes.

Real-Life Analogy

Imagine visiting a large hospital.

A hospital contains many departments:

- Reception
- Doctors
- Laboratory
- Pharmacy
- Billing
- Insurance

If every patient had to visit each department individually and understand the complete process, it would be confusing.

Instead, the patient simply visits the reception desk.

The receptionist coordinates everything internally:

- Schedules the doctor
- Sends lab requests
- Handles billing
- Directs the patient to the pharmacy

The patient only interacts with one person.

The receptionist acts as the Facade.

Another Real-Life Analogy

Imagine a company with thousands of employees.

A Director wants to assign work but doesn't know every employee's expertise.

Without any management layer, the Director would need to:

- Find the right employee
- Understand everyone's skills
- Assign work individually

This becomes difficult in a large organization.

Instead, the Director communicates only with the Manager.

The Manager:

- Understands the team's expertise
- Assigns work to the correct employee
- Coordinates the entire process

The Director only interacts with the Manager.

Here,

- Director → Client
- Manager → Facade
- Employees → Subsystems

Other Common Examples

Some common examples of the Facade Pattern include:

- ATM Machine
- Online Travel Booking Websites
- Banking Applications
- Video Player Applications
- Mobile Payment Apps
- API Gateway in Microservices
- Smart Home Controller
- Home Theater Remote

In all these cases, the user interacts with one simple interface while multiple systems work behind the scenes.

Problem Without Facade

Suppose a client wants to perform one business operation. Instead of calling one class, it may need to interact with multiple subsystem classes.

The client must:

- Create objects
- Understand dependencies
- Know the execution order
- Handle coordination between classes

As the system grows, every client becomes more complicated.

This leads to:

- Tight coupling
- Duplicate workflow logic
- Difficult maintenance
- Poor readability

Solution

Introduce a Facade class. The Facade exposes one simple method for the client.

Internally, the Facade:

- Creates or uses subsystem objects
- Calls them in the correct sequence

- Coordinates the complete workflow
- Returns the final result

The client never needs to know what happens internally.

Structure of the Facade Pattern

The Facade Pattern generally consists of three parts.

1. Client

The client is the user of the system. Instead of interacting with multiple subsystem classes, the client communicates only with the Facade. The client remains simple and independent of the internal implementation.

2. Facade

The Facade acts as the single entry point.

Its responsibilities include:

- Receiving client requests
- Coordinating subsystem classes
- Executing operations in the correct order
- Returning the final response

The Facade does not implement new business logic. Its primary responsibility is to simplify interaction with existing subsystem classes.

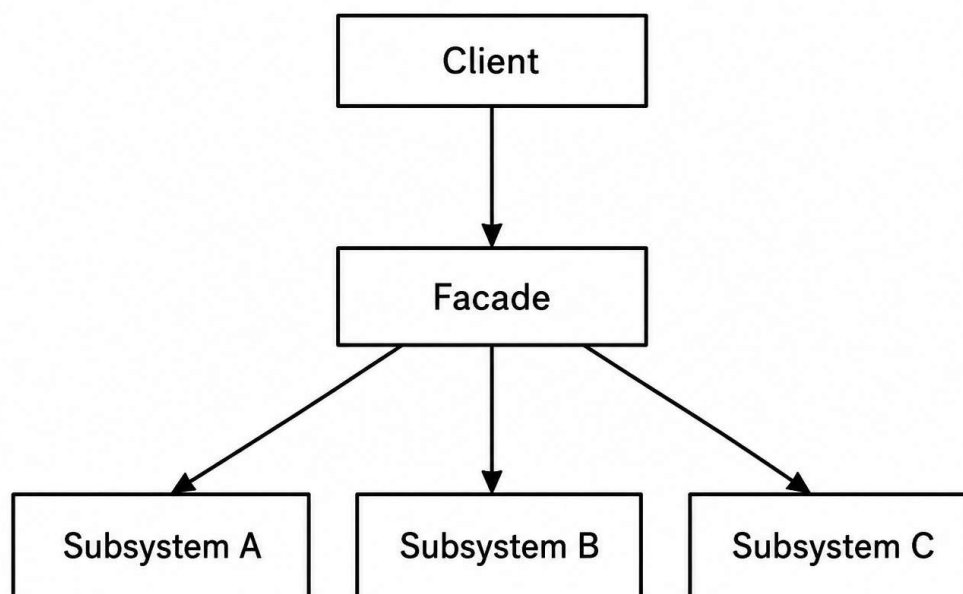
3. Subsystems

Subsystems are the classes that perform the actual work. Each subsystem has its own responsibility.

Examples include:

- Database operations
- Payment processing
- Report generation
- Notification handling
- Authentication
- File operations

The subsystems are usually unaware of the Facade. They continue performing their individual responsibilities independently.



The client only communicates with the Facade. The Facade communicates with all subsystem classes.

Implementation of Travel Booking System

Imagine you're building a travel booking application like **Expedia** or **MakeMyTrip**. When a customer books a vacation package, several independent systems need to work together.

The booking process involves:

- Booking the flight
- Reserving the hotel
- Booking a taxi
- Purchasing travel insurance
- Processing the payment
- Sending a confirmation email

Each of these tasks is handled by a different subsystem. If the client had to interact with every subsystem individually, the booking process would become complex and difficult to maintain.

Instead, we'll create a **TravelBookingFacade** that performs all these operations through a single method call.

1. Identify the Subsystems (Actual Components That Perform Individual Tasks) - Each subsystem has one responsibility.

(i) Create FlightService class - This class is responsible for booking flight tickets.

```
public class FlightService {  
    public void bookFlight(String source, String destination) {
```



```
        System.out.println("Flight booked from " + source + " to " +  
destination);  
    }  
}
```

(ii) Create HotelService class - This class reserves the hotel for the customer.

```
public class HotelService {  
    public void bookHotel(String city) {  
        System.out.println("Hotel booked in " + city);  
    }  
}
```

(iii) Create TaxiService class - This class books airport pickup and local transportation.

```
public class TaxiService {  
    public void bookTaxi(String city) {  
        System.out.println("Taxi booked in " + city);  
    }  
}
```

(iv) Create InsuranceService class - This class purchases travel insurance.

```
public class InsuranceService {  
    public void purchaseInsurance() {  
        System.out.println("Travel insurance purchased.");  
    }  
}
```

(v) Create PaymentService class - This class processes the payment.

```
public class PaymentService {  
    public void makePayment(double amount) {  
        System.out.println("Payment of ₹" + amount + "  
completed.");  
    }  
}
```

(vi) Create EmailService class - This class sends the booking confirmation email.

```
public class EmailService {  
    public void sendConfirmation() {  
        System.out.println("Booking confirmation email sent.");  
    }  
}
```

2. Create the Facade Class (Combines Everything) - Instead of requiring the client to interact with six different services, the Facade provides one simple method called **bookVacation()**, which internally coordinates the complete booking process.

```
public class TravelBookingFacade {  
    private FlightService flightService;  
    private HotelService hotelService;  
    private TaxiService taxiService;  
    private InsuranceService insuranceService;  
    private PaymentService paymentService;  
    private EmailService emailService;
```

```

public TravelBookingFacade() {
    flightService = new FlightService();
    hotelService = new HotelService();
    taxiService = new TaxiService();
    insuranceService = new InsuranceService();
    paymentService = new PaymentService();
    emailService = new EmailService();
}

public void bookVacation(String source, String destination,
double amount) {
    flightService.bookFlight(source, destination);
    hotelService.bookHotel(destination);
    taxiService.bookTaxi(destination);
    insuranceService.purchaseInsurance();
    paymentService.makePayment(amount);
    emailService.sendConfirmation();
    System.out.println("Vacation package booked
successfully.");
}
}

```

3. Client Uses the Facade (Instead of Directly Using Subsystems)

Now the client doesn't need to know anything about flights, hotels, taxis, insurance, payment, or email services. The client simply requests a vacation booking.

```

public class Main {
    public static void main(String[] args) {
        // Create the Facade
        TravelBookingFacade travelFacade = new
TravelBookingFacade();
    }
}

```

```
// Book the complete vacation package
travelFacade.bookVacation("Delhi", "Goa", 35000);
}
}
```

```
/* *
```

Output :

Flight booked from Delhi to Goa

Hotel booked in Goa

Taxi booked in Goa

Travel insurance purchased.

Payment of ₹35000.0 completed.

Booking confirmation email sent.

Vacation package booked successfully.

```
* */
```

What Happens Without the Facade?

The client would have to perform all these operations manually.

```
FlightService flightService = new FlightService();
```

```
HotelService hotelService = new HotelService();
```

```
TaxiService taxiService = new TaxiService();
```

```
InsuranceService insuranceService = new InsuranceService();
```

```
PaymentService paymentService = new PaymentService();
```

```
EmailService emailService = new EmailService();
```

```
flightService.bookFlight("Delhi", "Goa");
```

```
hotelService.bookHotel("Goa");
```

```
taxiService.bookTaxi("Goa");  
insuranceService.purchaseInsurance();  
paymentService.makePayment(35000);  
emailService.sendConfirmation();
```

The client must:

- Create every subsystem object.
- Know the correct order of execution.
- Coordinate all services manually.

As more services are added (such as visa booking, activity booking, coupon application, reward points, or SMS notifications), the client code becomes increasingly complex.

The **TravelBookingFacade** hides all this complexity by exposing a single method:

```
travelFacade.bookVacation("Delhi", "Goa", 35000);
```

The client only communicates with the Facade, while the Facade coordinates all the underlying subsystem classes. This is the essence of the Facade Pattern.

Implementation of E-commerce Order Placement

Imagine you're building an e-commerce application like **Amazon** or **Flipkart**. When a customer clicks the "**Place Order**" button, multiple independent services need to work together to complete the order.

The order placement process involves:

- Reserving inventory
- Processing payment
- Applying coupons
- Creating shipment
- Generating invoice
- Sending confirmation email
- Adding loyalty points

Each of these tasks is handled by a separate subsystem. If the controller had to interact with every service directly, it would become large, tightly coupled, and difficult to maintain.

Instead, we'll create an **OrderFacade** that coordinates all these operations through a single method call.

1. Identify the Subsystems (Actual Components That Perform Individual Tasks) - Each subsystem performs one specific responsibility.

(i) InventoryService - This service checks product availability and reserves the ordered items.

```
public class InventoryService {  
    public void reserveItems() {  
        System.out.println("Inventory reserved.");  
    }  
}
```

(ii) PaymentService - This service processes the customer's payment.

```
public class PaymentService {  
    public void makePayment(double amount) {  
        System.out.println("Payment of ₹" + amount + "  
completed.");  
    }  
}
```

(iii) CouponService - This service validates and applies any available coupon.

```
public class CouponService {  
    public void applyCoupon(String couponCode) {  
        System.out.println("Coupon " + couponCode + " applied.");  
    }  
}
```

(iv) ShippingService - This service creates the shipment for the order.

```
public class ShippingService {  
    public void createShipment() {  
        System.out.println("Shipment created.");  
    }  
}
```

(v) InvoiceService - This service generates the invoice.

```
public class InvoiceService {  
    public void generateInvoice() {
```

```
        System.out.println("Invoice generated.");
    }
}
```

(vi) NotificationService - This service sends the order confirmation email.

```
public class NotificationService {
    public void sendEmail() {
        System.out.println("Order confirmation email sent.");
    }
}
```

(vii) LoyaltyService - This service rewards the customer with loyalty points.

```
public class LoyaltyService {
    public void addRewardPoints() {
        System.out.println("Reward points added to customer account.");
    }
}
```

2. Create OrderRequest - Instead of passing multiple parameters, we'll encapsulate the order information inside an OrderRequest object.

```
public class OrderRequest {
    private double amount;
    private String couponCode;
```



```

public OrderRequest(double amount, String couponCode) {
    this.amount = amount;
    this.couponCode = couponCode;
}

public double getAmount() {
    return amount;
}

public String getCouponCode() {
    return couponCode;
}
}

```

3. Create the Facade Class - (Combines Everything) - Instead of requiring the controller to interact with seven different services, the Facade exposes one simple method called **placeOrder()**, which internally coordinates the complete order placement workflow.

```

public class OrderFacade {
    private InventoryService inventoryService;
    private PaymentService paymentService;
    private CouponService couponService;
    private ShippingService shippingService;
    private InvoiceService invoiceService;
    private NotificationService notificationService;
    private LoyaltyService loyaltyService;

    public OrderFacade() {
        inventoryService = new InventoryService();
        paymentService = new PaymentService();
        couponService = new CouponService();
        shippingService = new ShippingService();
        invoiceService = new InvoiceService();
        notificationService = new NotificationService();
    }
}

```

```

        loyaltyService = new LoyaltyService();
    }

    public void placeOrder(OrderRequest request) {
        inventoryService.reserveItems();
        couponService.applyCoupon(request.getCouponCode());
        paymentService.makePayment(request.getAmount());
        shippingService.createShipment();
        invoiceService.generateInvoice();
        notificationService.sendEmail();
        loyaltyService.addRewardPoints();
        System.out.println("Order placed successfully.");
    }
}

```

4. Client Uses the Facade - The controller doesn't need to know how inventory, payment, shipping, invoices, notifications, or loyalty points work. It simply delegates the request to the Facade.

```

public class Main {
    public static void main(String[] args) {
        OrderFacade orderFacade = new OrderFacade();
        OrderRequest request = new OrderRequest(2499,
"SAVE500");
        orderFacade.placeOrder(request);
    }
}

```

/* *

Output :

Inventory reserved.

Coupon 'SAVE500' applied.

Payment of ₹2499.0 completed.

*Shipment created.
Invoice generated.
Order confirmation email sent.
Reward points added to customer account.
Order placed successfully.*

* */

Employee Onboarding System

When HR hires a new employee, several systems need to work together before the employee can start working.

The onboarding process involves:

- Creating the employee record
- Creating the company email account
- Allocating a laptop
- Granting access to internal applications
- Registering the employee in payroll
- Sending a welcome email

Each of these tasks is handled by a different subsystem. Instead of the HR application interacting with every subsystem individually, we'll create an **EmployeeOnboardingFacade** that coordinates the complete onboarding process through a single method call.

1. Create EmployeeRequest

```
public class EmployeeRequest {
```

```
private String employeeName;  
private String email;  
private String department;  
  
public EmployeeRequest(String employeeName, String email,  
String department) {  
    this.employeeName = employeeName;  
    this.email = email;  
    this.department = department;  
}  
  
public String getEmployeeName() {  
    return employeeName;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public String getDepartment() {  
    return department;  
}  
}
```

2. Create EmployeeService - Creates the employee record.

```
public class EmployeeService {  
    public void createEmployee(EmployeeRequest request) {  
        System.out.println("Employee record created for " +  
request.getEmployeeName());  
    }  
}
```

3. Create EmailService - Creates the official company email account.

```
public class EmailService {  
    public void createEmailAccount(EmployeeRequest request) {  
        System.out.println("Company email account created: " +  
request.getEmail());  
    }  
}
```

4. Create AssetService - Allocates the company laptop.

```
public class AssetService {  
    public void allocateLaptop(EmployeeRequest request) {  
        System.out.println("Laptop allocated to " +  
request.getEmployeeName());  
    }  
}
```

5. Create AccessService - Grants access to company applications.

```
public class AccessService {  
    public void grantAccess(EmployeeRequest request) {  
        System.out.println("Access granted to Jira, GitHub and  
VPN.");  
    }  
}
```

6. Create PayrollService - Registers the employee in payroll.

```
public class PayrollService {
```

```
    public void registerEmployee(EmployeeRequest request) {  
        System.out.println("Employee registered in payroll.");  
    }  
}
```

7. Create NotificationService - Sends the welcome email.

```
public class NotificationService {  
    public void sendWelcomeEmail(EmployeeRequest request) {  
        System.out.println("Welcome email sent to " +  
request.getEmail());  
    }  
}
```

8. Create the Façade - Instead of requiring HR to interact with multiple systems, the Facade coordinates the complete onboarding workflow.

```
public class EmployeeOnboardingFacade {  
    private EmployeeService employeeService;  
    private EmailService emailService;  
    private AssetService assetService;  
    private AccessService accessService;  
    private PayrollService payrollService;  
    private NotificationService notificationService;  
  
    public EmployeeOnboardingFacade() {  
        employeeService = new EmployeeService();  
        emailService = new EmailService();  
        assetService = new AssetService();  
        accessService = new AccessService();  
        payrollService = new PayrollService();  
        notificationService = new NotificationService();  
    }  
}
```

```

public void onboardEmployee(EmployeeRequest request) {
    employeeService.createEmployee(request);
    emailService.createEmailAccount(request);
    assetService.allocateLaptop(request);
    accessService.grantAccess(request);
    payrollService.registerEmployee(request);
    notificationService.sendWelcomeEmail(request);
    System.out.println("Employee onboarding completed
successfully.");
}
}

```

9. Client Uses the Facade

```

public class Main {
    public static void main(String[] args) {
        EmployeeRequest request = new EmployeeRequest("John
Doe", "john.doe@company.com", "Engineering");
        EmployeeOnboardingFacade facade = new
EmployeeOnboardingFacade();
        facade.onboardEmployee(request);
    }
}

```

/* *

Output :

*Employee record created for John Doe
Company email account created: john.doe@company.com
Laptop allocated to John Doe
Access granted to Jira, GitHub and VPN.
Employee registered in payroll.
Welcome email sent to john.doe@company.com
Employee onboarding completed successfully.*

Characteristics of the Facade Pattern

- Provides a single entry point to a complex system.
- Simplifies client interaction.
- Hides implementation details.
- Coordinates multiple subsystem classes.
- Promotes loose coupling between the client and subsystems.
- Improves code readability.
- Makes workflows easier to maintain.
- Does not replace subsystem classes.
- Does not introduce new business functionality.
- Only simplifies existing functionality.

Benefits of the Facade Pattern

1. Reduces Complexity

Clients no longer need to understand the internal working of multiple subsystem classes.

2. Promotes Loose Coupling

Clients depend only on the Facade instead of many subsystem classes. This reduces dependencies across the application.

3. Improves Readability

Client code becomes shorter, cleaner, and easier to understand.

4. Centralizes Workflow Logic

The complete workflow exists in one place.

If the sequence changes, only the Facade needs to be updated.

5. Easier Maintenance

Changes inside subsystem classes rarely affect client code.

Only the Facade may require updates.

6. Better Layered Architecture

The Facade acts as an intermediate layer between clients and complex subsystems.

This leads to cleaner architecture.

7. Encourages Reusability

Different clients can reuse the same Facade instead of implementing identical workflows repeatedly.

Drawbacks of the Facade Pattern

1. Can Become a God Object

If too many responsibilities are added, the Facade can become excessively large and difficult to maintain.

2. Additional Layer

For small systems, introducing a Facade may add unnecessary abstraction.

3. May Hide Useful Functionality

Advanced clients may require direct access to subsystem classes for specialized operations.

4. Requires Maintenance

Whenever subsystem workflows change significantly, the Facade may also need updates.

When Should We Use the Facade Pattern?

Use the Facade Pattern when:

- A system contains many interacting classes.
- Clients repeatedly perform the same sequence of operations.
- You want to hide implementation details.
- You want to simplify a complex API.
- Multiple clients use the same workflow.
- You want to reduce coupling between clients and subsystem classes.
- You want to expose a cleaner interface to the rest of the application.

When Should We Avoid the Facade Pattern?

Avoid the Facade Pattern when:

- The system is already simple.
- Only one or two classes are involved.
- Clients require complete control over subsystem classes.
- Introducing another layer would only increase unnecessary complexity.

Difference Between Facade and Adapter

Facade

Simplifies a complex system.

Provides a new simplified interface.

Focuses on reducing complexity.

Works with multiple subsystem classes.

Adapter

Makes incompatible interfaces work together.

Converts one interface into another.

Focuses on compatibility.

Usually works between two incompatible classes.

Difference Between Facade and Abstract Factory

Facade Pattern

Structural Design Pattern

Simplifies interaction with a complex system.

Focuses on coordinating operations.

Hides complexity of using subsystem classes.

Uses existing objects.

Provides one simplified interface.

Abstract Factory Pattern

Creational Design Pattern

Creates families of related objects.

Focuses on object creation.

Hides complexity of creating objects.

Creates new objects.

Provides factories for object creation.

A simple way to remember the difference is:

- Abstract Factory creates objects.
- Facade uses existing objects.

What is UML?

UML (Unified Modeling Language) is a standard way to visually represent the design of a software system. It helps developers, architects, and stakeholders understand the structure and behaviour of an application before writing code.

UML is widely used in:

- Low-Level Design (LLD)
- High-Level Design (HLD)
- System documentation
- Software architecture discussions
- Technical interviews

Types of UML Diagrams

UML diagrams are mainly divided into two categories:

1. Structural Diagrams

These describe the static structure of the system.

Examples:

- Class Diagram (Most important for LLD)
- Object Diagram
- Package Diagram
- Component Diagram
- Deployment Diagram

2. Behavioural Diagrams

These describe how objects interact and behave.

Examples:

- Sequence Diagram (Most important for LLD)
- Use Case Diagram
- Activity Diagram
- State Machine Diagram
- Communication Diagram

For interviews, the two most important UML diagrams are:

- Class Diagram
- Sequence Diagram

Class Diagram

A Class Diagram represents the static structure of a system.

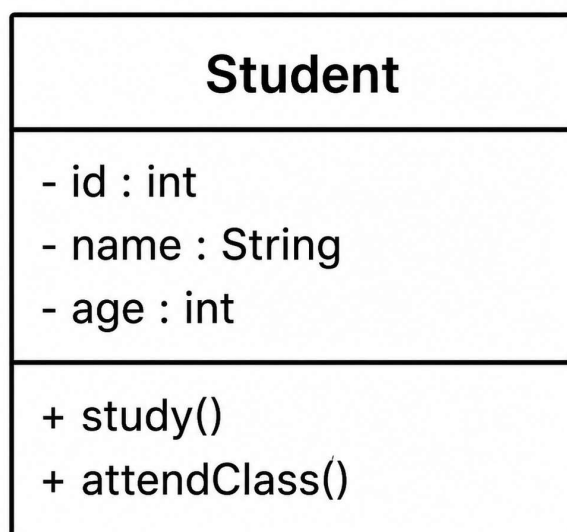
It shows:

- Classes
- Attributes
- Methods
- Relationships between classes

Think of it as the blueprint of your code.

Basic Structure of a Class

A class is represented using three sections.



Top section

- Class Name

Middle section

- Attributes (Variables)

Bottom section

- Methods (Functions)

Visibility Symbols

Symbol	Meaning	Accessible From
+	Public	Everywhere
-	Private	Inside same class only
#	Protected	Same class + Child classes
~	Package Default	Same package

Example :

```
- name : String
+ getName() : String
# calculateSalary()
~ printDetails()
```

Attributes

Attributes represent data members.

Example -

- id : int
- name : String
- salary : double

General syntax

visibility name : datatype

Example

- email : String
- + balance : double

Methods

Methods represent behavior.

Example-

- + deposit(amount : double)
- + withdraw(amount : double)
- + getBalance() : double

General syntax

visibility method(parameters) : returnType

Relationship Types

Relationships show how classes are connected. There are six major relationships :

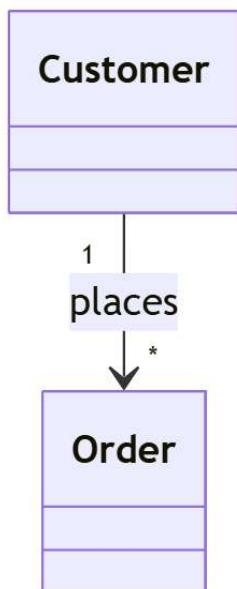
1. Association

One object knows or uses another object. It is the most common relationship.

Example

A Customer places Orders. A customer can place many orders.

```
class Customer {  
    Order order;  
}
```



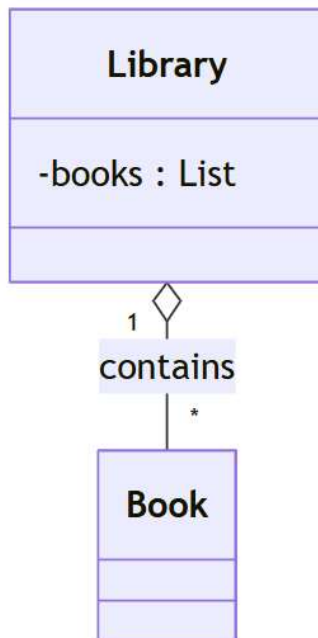
2. Aggregation

Has-a relationship with weak ownership. The child object can exist even if the parent is destroyed.

Example

Books can exist without a Library. If Library is deleted, Books still exist.

```
class Library {  
    List<Book> books;  
}
```



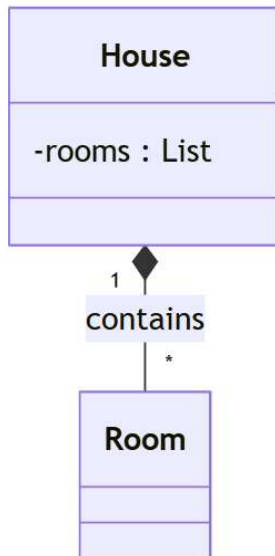
3. Composition

Strong Has-a relationship. Child cannot exist without Parent.

Example

A **House** has **Rooms**. If the **House** is deleted, all its **Rooms** are also deleted.

```
class House {  
    List<Room> rooms = new ArrayList<>();  
}
```



4. Inheritance (Generalization)

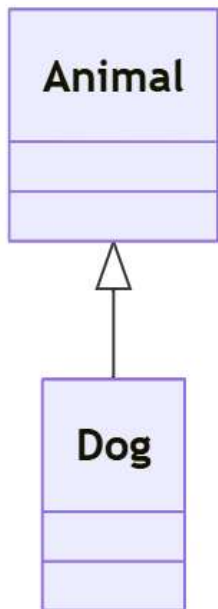
Represents an **is-a** relationship. A child class inherits the properties and behaviour of the parent class.

Example

Dog is an Animal.

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```



5. Dependency

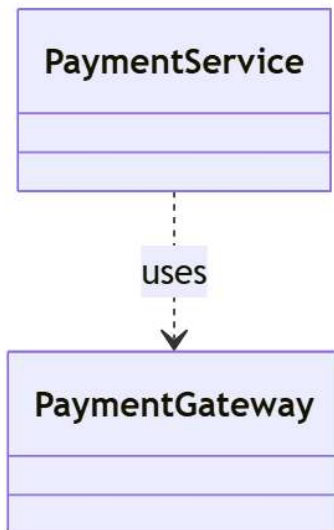
One class **temporarily uses** another class. No permanent reference to the other object is stored.

Example

A **PaymentService** uses a **PaymentGateway** to process a payment.

```
class PaymentService {
    void pay(PaymentGateway gateway) {
        gateway.process();
    }
}
```

Gateway is only used inside the method.



6. Interface Implementation (Realization)

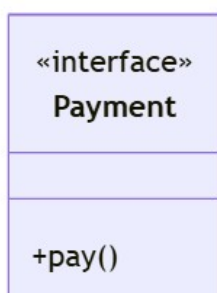
A class implements an interface.

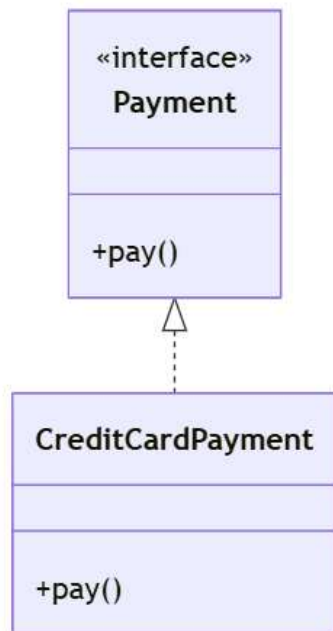
Example

CreditCardPayment implements the Payment interface.

```
interface Payment {  
    void pay();  
}
```

```
class CreditCardPayment implements Payment {  
    @Override  
    public void pay() {  
    }  
}
```





Multiplicity

Multiplicity shows **how many objects participate in a relationship**.

Multiplicity Meaning

1	Exactly one
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many

Sequence Diagram

A Sequence Diagram is a UML Behavioral Diagram that shows how different objects communicate with each other over time to complete a task.

Unlike a Class Diagram, which shows the structure of a system, a Sequence Diagram shows the runtime behavior.

It answers questions like:

- Who starts the request?
- Which object is called first?
- Which object calls the next object?
- What is the order of execution?
- What response comes back?

Think of it as the timeline of a feature.

Example:

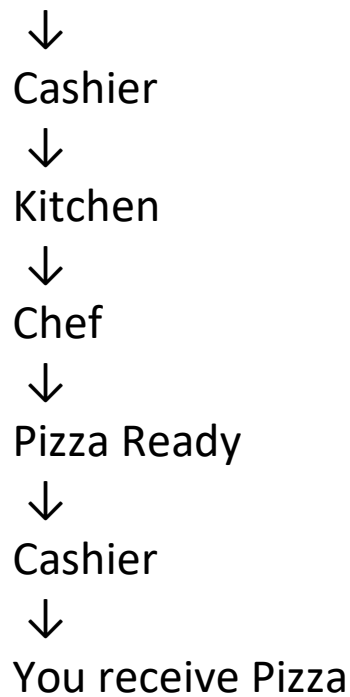
Customer places an order on Amazon. The sequence diagram shows every method call from clicking the button until the order is successfully placed.

Real World Example

Suppose you order a pizza.

The steps are:

You



Notice that everyone performs one task and then passes control to the next person. A Sequence Diagram works exactly like this.

Main Components

1. Actor

An **Actor** is the person or external system that starts or triggers the process shown in the sequence diagram. The Actor is **outside the system** and interacts with it.

Examples

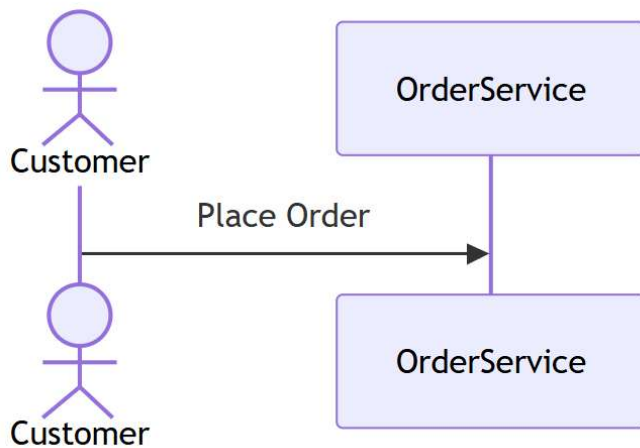
- Customer
- User
- Admin
- ATM User
- Doctor

- Payment Gateway



Example

Suppose a customer places an order.



Here:

- **Customer** is the **Actor** because it initiates the process.
- **OrderService** is a participant inside the system that handles the request.

Remember: The Actor always appears on the **left side** of a sequence diagram and starts the interaction.

2. Object (Participant)

A **Participant** is an object, class, or service that takes part in completing the request.

Each participant has its own **lifeline** showing its existence during the interaction.

Examples

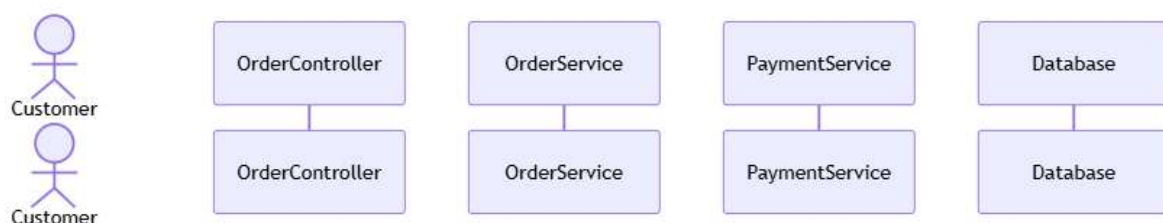
- OrderController
- OrderService
- InventoryService
- PaymentService
- Database

Note: The **Customer** is usually an **Actor**, not a participant, because it starts the interaction from outside the system.

A **Lifeline** is the vertical dashed line below an Actor or Participant. It represents the existence of that object while the interaction is taking place. Every Actor and Participant has its own lifeline.

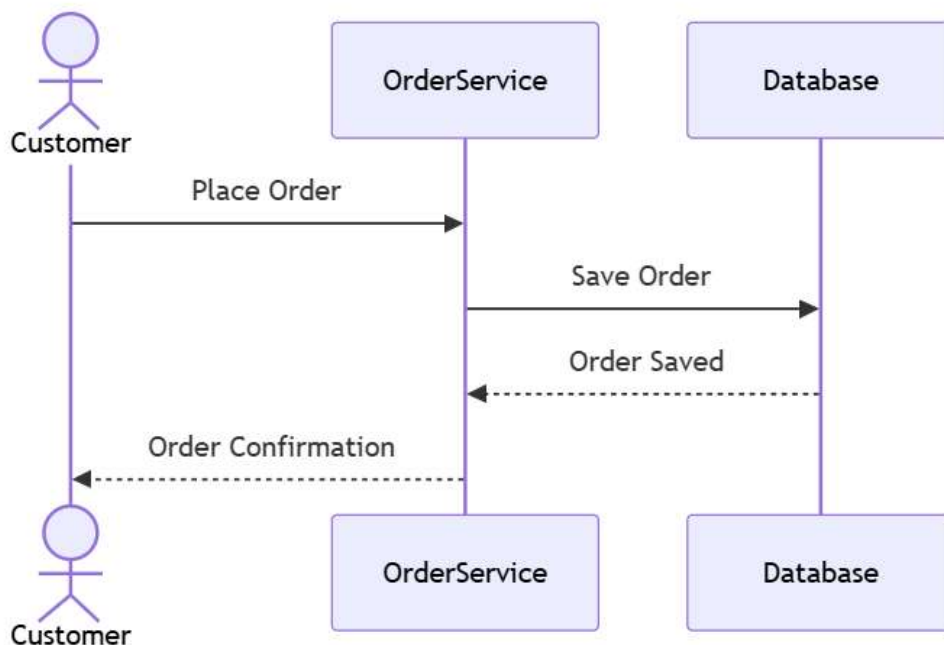
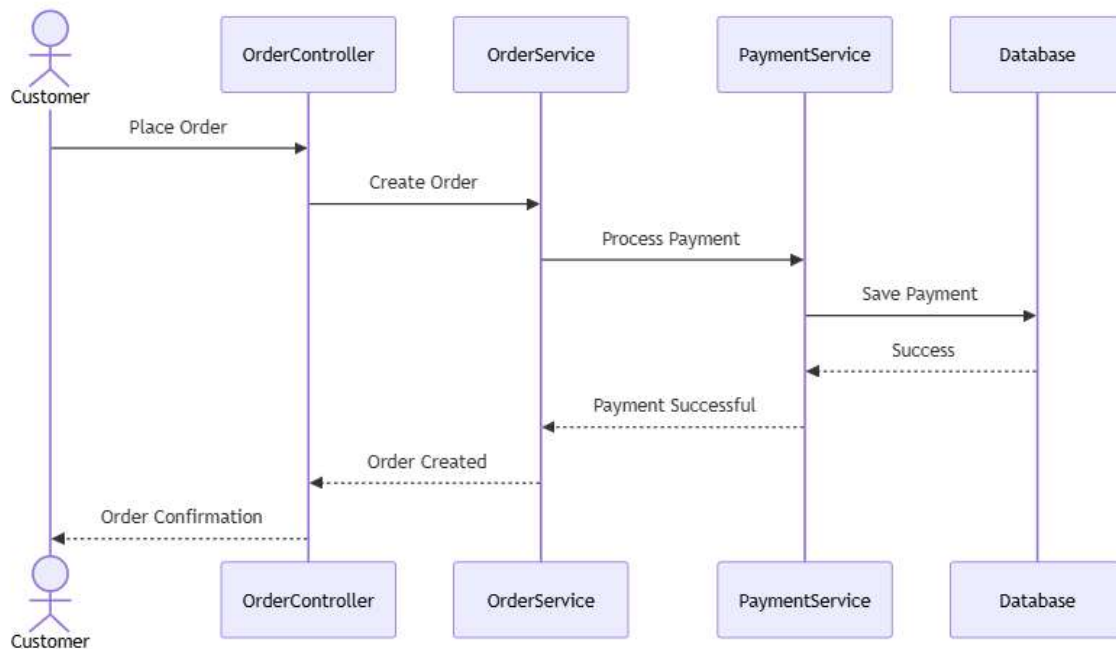
The vertical line (|) represents the **Lifeline**.

Diagram (Multiple Objects)



Example

A customer places an order.



Remember

- **Actor** → Starts the process (outside the system).
- **Participant** → Objects or services inside the system that collaborate

to complete the request.

- Every participant has its own **lifeline**.

3. Activation Bar

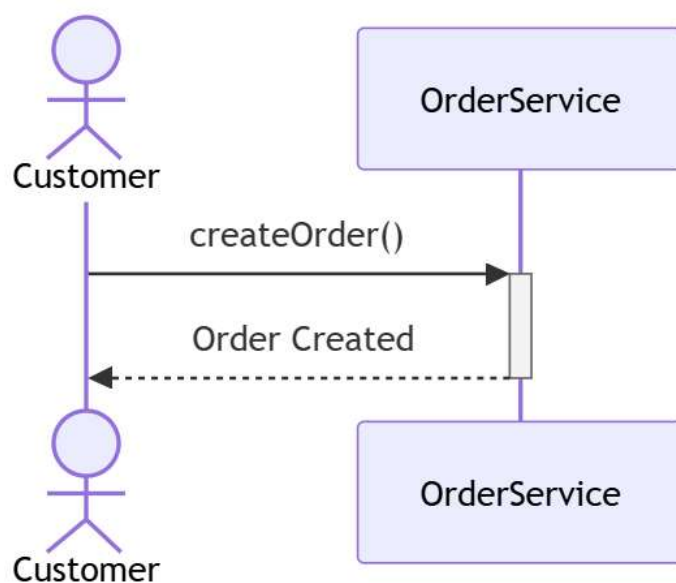
An **Activation Bar** is the thin rectangle drawn on a participant's lifeline. It shows the period during which the participant is **actively executing a method or processing a request**.

Meaning object is currently executing some code. The shaded rectangle (■■■■) represents the **Activation Bar**.

Example

Suppose OrderService is creating an order.

The activation bar indicates that **OrderService** is executing the **createOrder()** method. Once execution finishes, the activation ends.



4. Messages

Objects communicate with each other by sending **messages**. A message usually represents a **method call** from one object to another.

Example

OrderService -----> PaymentService

This means:

OrderService calls PaymentService.

There are three common types of messages.

Synchronous Message

The caller waits until the called object finishes executing.

The caller cannot continue until it receives a response.

Think Like

Customer: Can you process my payment?

(waiting...)

PaymentService: Done.

The customer waits for the payment to finish.

Real Example

Customer



OrderService



PaymentService

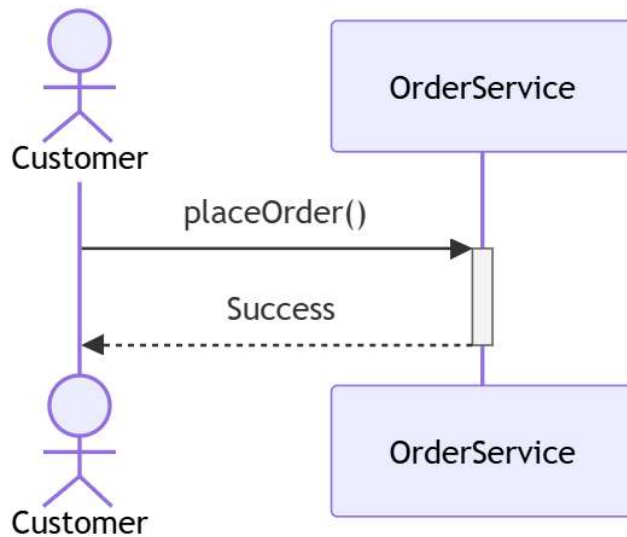


Payment Completed



Return Success

Diagram



Remember

- Caller waits.
- Used for database calls, payment processing, API calls, etc.

Asynchronous Message

The caller sends a request and immediately continues. It does **not** wait for the other object to finish.

Think Like

OrderService -> Send Email -> Continue processing...

Even if sending the email takes 10 seconds, the order is already completed.

Real Example

Order Placed



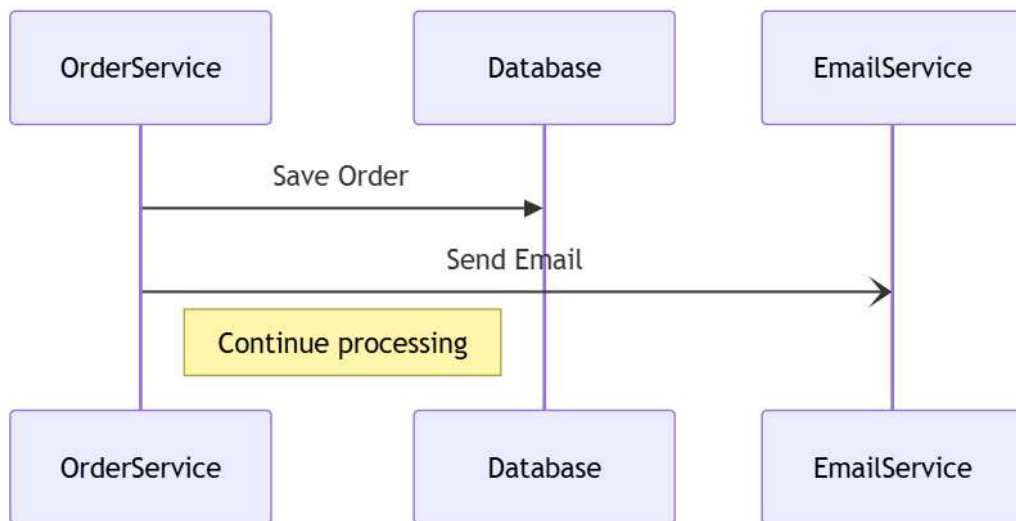
Save Order



Send Email



Continue Processing



Typical Examples

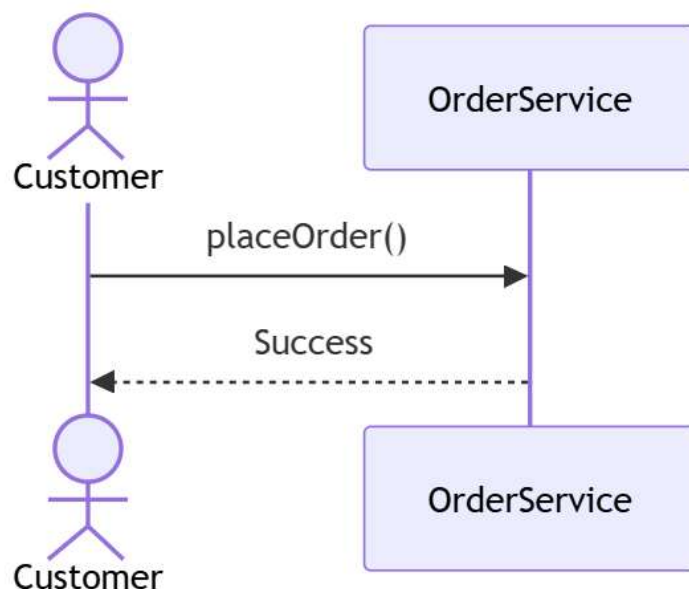
- Email
- SMS
- Push Notification
- Kafka Event
- RabbitMQ
- Background Job

Remember

- Caller does **not** wait.
- Execution continues immediately.

Return Message

A **Return Message** shows that a method has finished and control is returned to the caller. Return messages are usually shown using a **dashed arrow**.



Remember :

A return message indicates that the requested operation has completed.

In UML, the return arrow is usually dashed. Mermaid automatically renders `-->>` as a dashed return message.

Complete Sequence Diagram Example

Let's understand a real Amazon "**Place Order**" flow.

Step 1: Customer clicks Place Order

The customer initiates the process by clicking the **Place Order** button.

Step 2: Customer sends the request

The request is sent to the OrderController.

Customer → OrderController : placeOrder()

Step 3: Controller calls the service

The controller forwards the request to the OrderService.

OrderController → OrderService : createOrder()

Step 4: Service checks inventory

Before creating the order, the service verifies that the product is in stock.

OrderService → InventoryService : checkStock()

InventoryService → OrderService : Stock Available

Step 5: Service processes payment

If stock is available, the payment is processed.

OrderService → PaymentService : processPayment()

PaymentService → OrderService : Payment Success

Step 6: Store the order

After successful payment, the order is saved in the database.

OrderService → Database : saveOrder()

Database → OrderService : Order Saved

Step 7: Send confirmation email (Asynchronous)

The order confirmation email is sent asynchronously. The OrderService does **not** wait for the email to be sent.

OrderService -->> EmailService : sendConfirmation()

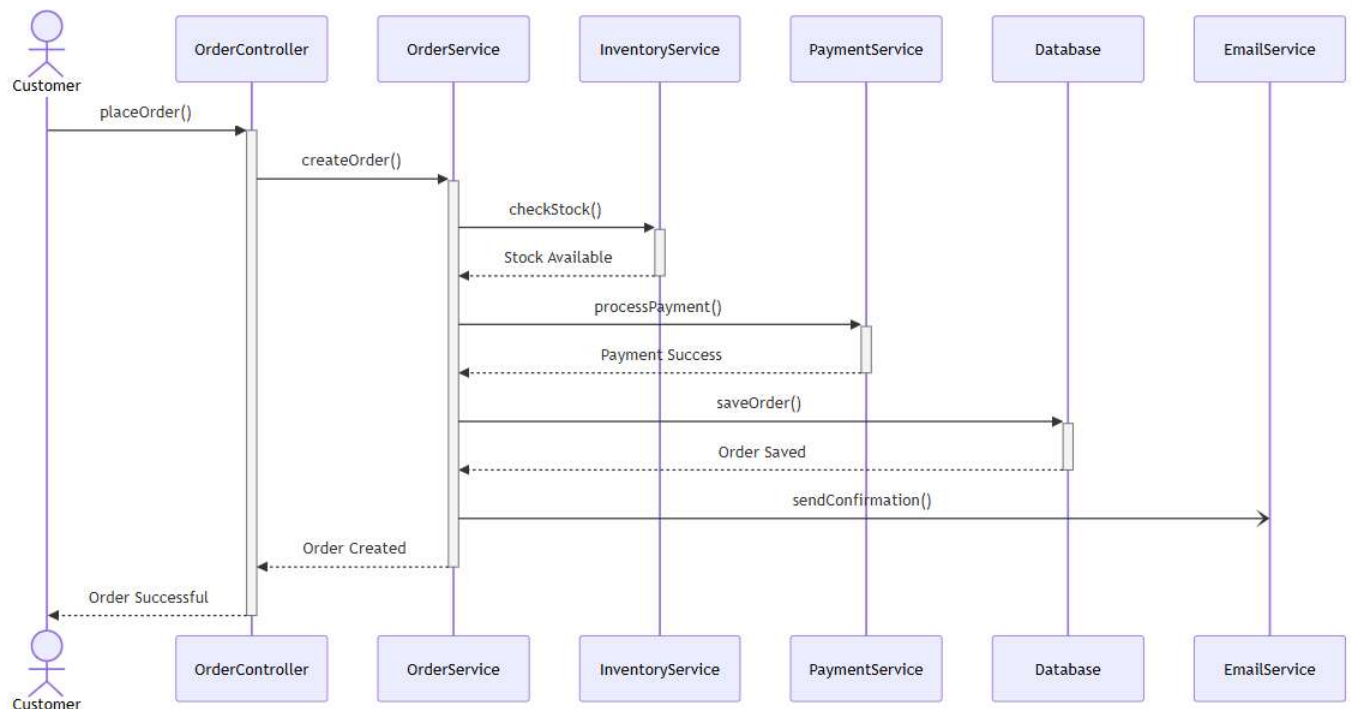
Step 8: Return success

Finally, the response is returned to the customer.

OrderService → OrderController : Order Created

OrderController → Customer : Order Successful

Complete Sequence Diagram



How to Draw a Sequence Diagram

Whenever you're asked to draw a sequence diagram:

1. Identify the actor (who starts the process?).
2. List all participants (controllers, services, databases, external systems).
3. Draw a lifeline for each participant.
4. Draw messages in the exact order they occur.
5. Add activation bars while objects execute methods.
6. Use solid arrows for synchronous calls.
7. Use dashed arrows for return messages.
8. Use dashed/open arrows for asynchronous operations like emails or notifications.
9. Ensure time flows from top to bottom.

10. Verify the sequence matches the actual execution flow.

This approach makes it easy to explain features such as **"Place Order"**, **"Login"**, **"Book Ticket"**, or **"Process Payment"** in LLD interviews.